

Hoe kan data worden verzameld voor een dataset?

**AI-Regeltechniek
ELEEEP71**

**Thijs Sijssens 1011823
Niels Noordzij 1006224**

18 januari 2026



Inhoudsopgave

1	Introductie	3
2	Trainingsdata	4
2.1	Data Discovery	4
2.2	Data Augmentation	5
2.3	Data Generation	6
3	Validatie	7
3.1	K-Fold Cross	7
4	Simulatie Onderzoek	8
5	Dataset genereren	9
5.1		9
5.2	Behavior Cloning	9
5.3	Reinforcement Learning vs Behavior Cloning	10
6	Conclusie en Toepassingen	11
7	Figuren	12
8	Change-log	16

Lijst van figuren

1	Unity omgeving.	12
2	Gedeeltelijke trainingsdata	13

1 Introductie

Een AI-model kan worden gebruikt om verschillende taken autonoom uit te voeren. Echter, een AI-model weet vanaf het eerste moment van creatie niet hoe het bepaalde taken moet uitvoeren en op basis van welke gegevens. Een AI-model moet getraind worden in het uitvoeren van een taak. Om dit te kunnen bereiken, moet er een training-set worden gemaakt. Een training-set is een grote verzameling van datapunten die overeenkomen met het daadwerkelijke systeem.

Voor dit project wordt een AI-model ontwikkeld dat de PID-regelaar zal vervangen in de Ping-Pong opstelling voor de cursus Digitale Systemen van de Hogeschool Rotterdam. Er zijn verschillende manieren om een training-set te ontwikkelen voor het trainen van een AI-model voor het aansturen van een PID-regelaar. De methodes waarin dit onderzoek naar gekeken is, zijn:

- De data uit de PID-regelaar van de Ping-Pong opstelling uitlezen en deze gebruiken
- Synthetic Data Generation

Dit onderzoek zal voornamelijk een literatuur onderzoek worden, met als doel om genoeg informatie te beschikken om een werkend embedded AI systeem te maken.

2 Trainingsdata

Het doel bij het vergaren van trainingsdata is het vinden van een geschikte dataset en, als deze niet beschikbaar is, het creëren van een geschikte dataset. Deze stappen worden ook al 'data discovery', 'data augmentation' en 'data generation' genoemd.

2.1 Data Discovery

Bij data discovery wordt er gezocht naar een geschikte dataset voor het doel van de AI. In dit tijdperk zijn er steeds meer verschillende datasets beschikbaar, dus het kiezen van een geschikte dataset is tegelijkertijd gemakkelijker, maar ook moeilijker geworden door het grote aantal keuzes. Er zijn twee verschillende stappen om een dataset te kunnen bemachtigen: 'Data Sharing' en 'Data Searching' [1].

Data Sharing Bij 'Data Sharing' worden datasets gemaakt door bedrijven of instituten die deze datasets ook publiceren. Het publiceren van een dataset kan op verschillende manieren worden gedaan. Zo kan een dataset worden gepubliceerd via een 'Collaborative Analysis'. Bij deze manier werken meerdere onderzoekers aan de analyse en verbetering van een dataset. Elke onderzoeker werkt aan zijn of haar dataset en plaatst deze op een later moment op een gezamenlijke DataHub. Een DataHub is een centrale server die, net als Git, werkt met versiebeheer. Een DataHub beschikt over verschillende standaardbibliotheken waarmee veelgebruikte data-processinghandelingen kunnen worden uitgevoerd [2]. Op deze manier kan iedere onderzoeker zijn of haar verbeterde dataset op de DataHub zetten, zodat deze samengevoegd kan worden tot één grote dataset.

Naast het gebruik van een DataHub bij 'Collaborative Analysis' kan een dataset ook op het internet worden geplaatst. Een veelgebruikte webservice is Google Fusion Tables[3]. Gebruikers kunnen hun datasets uploaden naar de servers, en deze zijn vanaf dat moment beschikbaar voor iedereen op het internet. Google Fusion Tables heeft ook verschillende tools waarmee de data visueel kan worden geanalyseerd en gefilterd.

Er wordt in de laatste paar jaren ook steeds meer gebruikgemaakt van de 'Collaborative and Web'-methode. Bij deze methode wordt een tool gebruikt die, naast het beheren van alle datasets, deze datasets kan delen met andere onderzoekers via het internet. Een tool die hiervoor gebruikt kan worden, is bijvoorbeeld Kaggle [4].

Data Searching Hierboven is besproken hoe datasets gedeeld kunnen worden, maar binnen een bedrijf kunnen er ontzettend veel datasets worden gemaakt en gedeeld. Om ervoor te zorgen dat andere onderzoekers toch een geschikte dataset kunnen vinden, wordt in dit soort bedrijven gebruikgemaakt van een 'Data Lake'. De definitie van een Data Lake is een verzameling van meerdere locaties met ruwe data die zijn beschreven met metadata en gecategoriseerd zijn. Daarnaast moet deze data te allen tijde beschikbaar zijn [5]. Het gebruik van een Data Lake zorgt ervoor dat het zoeken naar de juiste dataset efficiënter en accurater is. Het probleem is echter dat steeds meer bedrijven datasets gaan maken, wat ervoor zorgt dat een Data Lake uitgroeit tot een onbruikbare verzameling. Om dit probleem op te lossen, zijn er tools ontwikkeld die speciaal zijn ontworpen om grote hoeveelheden data te categoriseren. Één van deze tools is 'Google Data Search' [6].

Als er met behulp van de hierboven genoemde methoden een dataset gevonden is die niet geheel geschikt is voor de toepassing, kan deze dataset nog worden aangepast door middel van 'Data Augmentation' [1].

2.2 Data Augmentation

Als er een dataset is gevonden die beweert dat deze doet wat je verwacht, kan het toch voorkomen dat er problemen met de dataset zijn. Er kan namelijk te veel data aanwezig zijn in een dataset. Wanneer een dataset te veel informatie bevat, kan het voorkomen dat er corrupte data in de dataset zijn verwerkt die moeilijk te traceren zijn. Dit kan ervoor zorgen dat het model meer moeite heeft om een voorspelling te maken [7].

Een ander probleem dat kan ontstaan, is dat er '*overfitting*' kan optreden. Bij overfitting is een model te veel getraind op de trainingsdata. Dit kan resulteren in het feit dat het model 100% van de tijd werkt op de trainingsdata, maar misschien maar 50% op de testdata [8][9]. Het kan ook voorkomen dat een dataset te weinig data bevat voor het trainen van een model. Om deze dataset op te vullen, kan gebruik worden gemaakt van Data Augmentation technieken.

Symbolic Augmentation Bij Symbolic Augmentation wordt gebruikgemaakt van een statistisch AI model om data te genereren. Het voordeel hiervan is dat het makkelijker is om verschillende stukken tekst aan te passen met het AI model. Deze methode is echter niet geschikt als er grote hoeveelheden tekst moeten worden aangepast [7].

Rule-based Augmentation Met Rule-based Augmentation worden er regels of programma's vooraf gedefinieerd, en aan de hand van deze regels of programma's wordt nieuwe geaugmenteerde data gecreëerd. Een voorbeeld van Rule-based Augmentation is 'Synonym Replacement' [7]. Bij Synonym Replacement wordt de tekst gescand om te zien of bepaalde woorden vervangen kunnen worden door een synoniem. Dit zorgt ervoor dat de dataset groter en robuuster wordt [10].

Mixup Augmentation Met Mixup Augmentation wordt oude data gemengd met nieuwe data om zo nog meer nieuwe data te creëren. Het gebruik van deze methode heeft in sommige onderzoeken bewezen dat deze goed werkt tegen overfitting [7].

2.3 Data Generation

Bij het trainen van een AI model is het dus van belang dat er een complete, robuuste dataset is. In sommige gevallen kan dit op het internet worden gevonden omdat het al door een andere persoon is gemaakt. Als dit niet het geval is, kan ervoor gekozen worden om een dataset te genereren. De data die wordt gegenereerd, wordt ook wel 'Synthetic Data' genoemd. Synthetische Data wordt gezien als data die is gecreëerd door een computeralgoritme of door een simulatie. Een van de belangrijkste redenen voor synthetische data is dat deze data geen privacygevoelige informatie bevat [11]. Het gebruik van een simulatie maakt het mogelijk om gemakkelijk nieuwe data te creëren, zonder dat de basisinformatie verloren gaat. Daarnaast wordt alle data onder dezelfde unieke omstandigheden gecreëerd [12].

Het ontwikkelen van een simulatie kan met verschillende programma's worden uitgevoerd. Het kiezen van een programma is afhankelijk van de soort simulatie die nodig is. Voor het ontwikkelen van een 3D simulatie kan bijvoorbeeld Blender[13] gebruikt worden. Voor het ontwikkelen van een simulatie die gebaseerd is op fysica, kan het programma Unity[14] worden gebruikt.

De andere manier om synthetische data te genereren, is door gebruik te maken van 'Generative AI'. Met Generative AI kan data worden gegenereerd door middel van het gebruik van al bestaande data. Een AI model traint zichzelf dan aan de hand van deze data, wat ook wel *unsupervised learning* wordt genoemd. Een andere methode die gebruikt kan worden, is Self-Supervised Learning. [11].

Een model dat gebruikmaakt van *Self-Supervised learning* probeert van een niet-gecategoriseerde dataset een gecategoriseerde dataset te maken door de structuur van de dataset te begrijpen. Dit kan gedaan worden door te kijken naar de relatie tussen verschillende datapunten die altijd samenkomen. Het voordeel van het gebruik van Self-Supervised Learning in tegenstelling tot Supervised Learning is dat de dataset niet door een mens gecategoriseerd hoeft te worden. Dit bespaart veel tijd en ook veel geld [15].

Een andere manier van datageneratie genereert geen dataset. Maar deze manier kijkt naar binnenkomende data en voor een en maakt een voorspelling aan de hand van deze data. Deze manier wordt ook wel *Online Learning* genoemd. Het voordeel van het gebruik van Online Learning is dat het model direct en efficiënt geüpdatet kan worden aan de hand van de nieuwe data. Online Learning is ook beter schaalbaar voor grotere projecten, omdat het geen grotere dataset nodig heeft om mee te beginnen [16].

3 Validatie

Als een AI model is getraind, moet het nog worden gevalideerd om er zeker van te zijn dat het model werkt zoals gepland. Omdat de populariteit van AI de laatste jaren enorm is gestegen, zijn er verschillende manieren ontwikkeld om de validiteit van een AI model te kunnen testen. Een manier om een AI model te valideren, is door gebruik te maken van '*k-Fold cross validation*'.

Er zijn veel verschillende manieren om de betrouwbaarheid van een model te testen. In dit onderzoek is er alleen gekeken naar de K-Fold Cross methode. De reden hiervoor is dat het een eenvoudige validatiemethode is die makkelijk te implementeren is.

3.1 K-Fold Cross

Bij het gebruik van de k-fold cross methode wordt de dataset opgedeeld in K aantal delen (folds), en deze folds dienen dan als trainingsdata en testdata. Daarna wordt één van de folds gebruikt als testdata, en de overige folds worden gebruikt als trainingsdata. Deze routine wordt herhaald totdat de routine K aantal keer heeft plaatsgevonden. Tijdens het trainen wordt de prestatie van het model gemeten. Aan het einde van de routines wordt het gemiddelde genomen van de gemeten prestaties en die waarde is de betrouwbaarheid van het model [17].

4 Simulatie Onderzoek

Zoals besproken in hoofdstuk [2.3] kan er voor het genereren van data gebruik worden gemaakt van een simulatie. Er zijn door de jaren heen verschillende simulatieprogramma's ontwikkeld die kunnen worden gebruikt voor verschillende doeleinden. Zo zijn er simulatieprogramma's ontwikkeld die kunnen worden gebruikt voor bijvoorbeeld het simuleren van bedrijfsprocessen of voor het simuleren van logistieke opstellingen [18].

Voor dit project is een geschikte fysische simulatiesoftware geselecteerd, die in staat is om zowel de gravitatiekracht (zwaartekracht) als de aerodynamische weerstandskrachten nauwkeurig te modelleren. De geselecteerde simulatiesoftware is Unity geworden. Unity heeft een zeer sterke physics engine die het mogelijk maakt om de hiervoor genoemde onderdelen te simuleren. Hierdoor kan de werkelijke opstelling vrij nauwkeurig worden nagebootst in de simulatiesoftware en kan aan de hand van de simulatie trainingsdata worden gegenereerd [14]. In figuur [1] is te zien hoe de testopstelling is nagebootst in Unity. In figuur [2] is een deel van het resultaat van de simulatie te zien.

5 Dataset genereren

5.1

Reinforcement Learning Bij reinforcement learning is een mogelijke optie om zelf datasets te creëren. Dit doet het AI programma door een bepaalde actie, en dan vooral de mate waarin deze correct wordt uitgevoerd, te koppelen aan een beloning. Deze beloning wordt door een mens ingesteld en wordt bepaald door te kijken naar wat het AI moet leren. Ook kunnen er meerdere beloningen voor verschillende criteria zijn. De (reeks van) inputs die de hoogste beloning krijgt, wordt hierna vaker herhaald en bij lage beloningen worden die inputs minder of niet herhaald. Bij een echt reinforcement learning systeem worden inputs getest door trail and error. Als voorbeeld; een AI die moet leren om een auto te besturen, zou een beloning kunnen krijgen als er niet van de weg gereden wordt, als de snelheid niet boven de snelheidslimiet komt, als de auto geen ongeluk veroorzaakt of/en als de auto comfortabel rijdt [19].

Voordelen van reinforcent learning zijn dat het goed werkt, vooral na genoeg training en bij het correct instellen van de beloning die de AI krijgt. In dit geval genereert het zijn eigen data en dus is het niet afhankelijk van het internet of externe datasets. Dit zorgt ervoor dat het als onafhankelijk systeem kan opereren, wat perfect is voor een embedded systeem.

Omdat het systeem werkt door middel van trail and error, kan het lang duren voordat het AI systeem de koppeling maakt tussen bepaalde inputs en een beloning. Dit kan eventueel verminderd worden door bijvoorbeeld al bestaande datasets toe te voegen. Helaas is dat ook meteen een ander groot nadeel: omdat dit vaak gebruikt wordt door kleinere systemen waarvan geen of weinig datasets te verkrijgen zijn, kan dit dus vaak niet ondervangen worden en moet het systeem vanaf nul beginnen met leren. Als laatste kan dit systeem alleen leren onder directe supervisie van een mens, omdat het AI systeem niet zelf de beloning criteria kan bepalen [20].

5.2 Behavior Cloning

Behavior cloning is een techniek waarbij de dataset bestaat uit een 'expert' die datgene doet wat een ai moet leren. Met expert wordt hier een mens of een al bestaand, AI-loos systeem bedoeld waaruit data te verkrijgen is. De AI probeert dan deze expert zo goed mogelijk na te bootsen. Deze techniek is behulpzaam als er geen data beschikbaar is, maar als er wel al een werkend fysiek product bestaat of een menselijke expert waaruit een dataset te verkrijgen is [21].

Als nu hetzelfde voorbeeld wordt gebruikt als bij reinforcement learning, een zelfrijdende auto, kan de dataset bestaan uit de beelden/sensoren die de omstandigheden van de auto detecteren en de inputs van de persoon die de auto bestuurd zoals stuurbewegingen, de stand van het rem- en gaspedaal, de knipperlichten en de snelheid van de auto. Als hier genoeg data van is, kan de AI de input van de bestuurder nabootsen en theoretisch gezien kan de AI dan zelf een auto besturen.

Het grote voordeel van behavior cloning is dat er geen trail and error nodig is en dus dat het leerproces makkelijker opstart. Het nadeel is dat het vergaren van data soms niet makkelijk is omdat er persé een 'expert' aanwezig moet zijn. Ook dit systeem kan compleet zonder het internet gebruikt worden [22].

5.3 Reinforcement Learning vs Behavior Cloning

Hoewel reinforcement learning en behavior cloning allebei in de juiste omstandigheden goed werken, zijn er toch een aantal verschillen:

- Behavior cloning heeft een werkend systeem of data van een menselijke expert nodig om van te leren, waarin reinforcement learning alleen iemand nodig heeft om de beloningsfactoren te bepalen.
- Behavior cloning werkt niet met trail and error. Dit zorgt ervoor dat het begin van het leerproces veel sneller gaat bij behavior cloning.
- Omdat behavior cloning gebruik maakt van data van een mens, zal het ook nooit iets doen wat de expert niet doet. Reinforcement learning daarentegen is vrij te doen wat nodig is om de beloning zo groot mogelijk te maken. Hierdoor is behavior cloning iets voorspelbaarder en beheersbaarder dan reinforcement learning.
- Omdat met trail and error er ook veel datasets minder nuttig zijn, zijn de gebruikte datasets groter bij reinforcement learning dan bij behavior cloning.

Er zijn ook een aantal overeenkomsten:

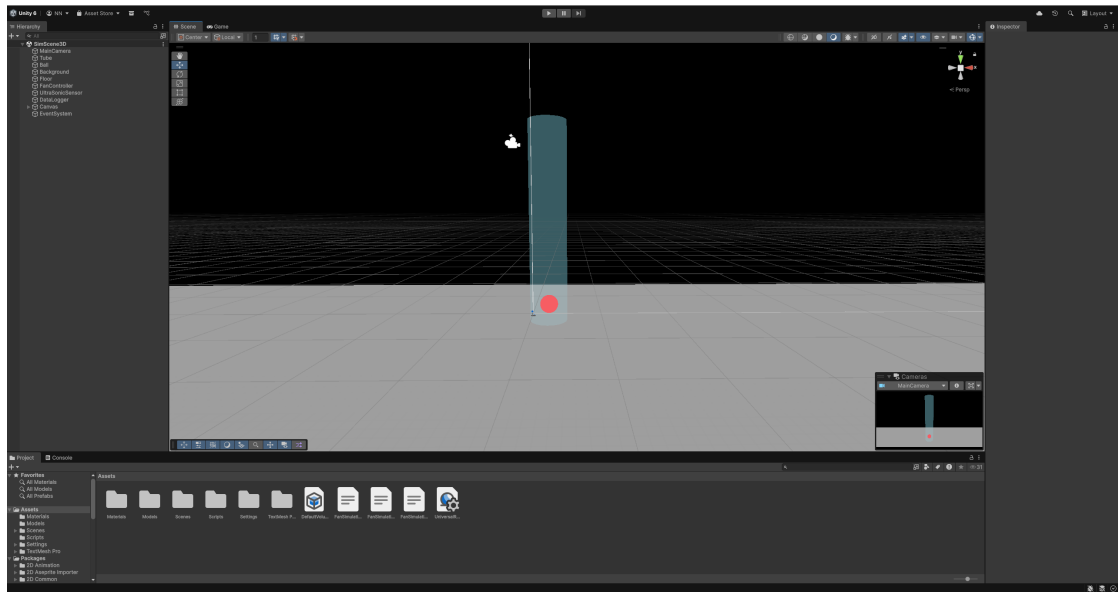
- Beide technieken maken vaak exclusief gebruik van hun eigen data en kunnen losgekoppeld van het internet opereren. Dit maakt het ideaal voor embedded systemen.
- Met beide technieken is, met voldoende data, een wekend AI model realiseerbaar.

Het fijne is dat het niet heel zwart wit is welke techniek gebruikt moet worden. In de werkelijkheid wordt vaak een combinatie gebruikt. Dit zorgt ervoor dat de beginfase van het leerproces versneld door de behavior cloning, waarna het AI model ook door kan leren zonder data van experts door reinforcement learning. In deze combinatie kunnen beide technieken elkaars zwaktes compenseren [23], [24], [25].

6 Conclusie en Toepassingen

Voor ons is het slim om een simulatie in Unity te maken. Hier is al ervaring mee binnen de groep en Unity heeft een goede physics engine waardoor de simulatie het meest overeenkomt met de werkelijke opstelling. Voor de bruikbaarheid van de datasets is deze overeenkomst erg van belang. Voor het genereren van datasets wordt behavior cloning gebruikt omdat er in dit project al een werkende fysieke opstelling is waardoor dit makkelijk toepasbaar is. Daar boven op scheelt dit trainingstijd waardoor de kans op tijdnood het kleinst is. In het geval dat daarna nog tijd is kan reinforcement learning ook toegepast worden om het AI model beter te trainen. Om het AI model te valideren wordt k-fold cross gebruikt.

7 Figuren



Figuur 1: Unity omgeving.

Time,FanPower,MeasuredHeight,BallVelocity,TargetHeight
0,0.12943,177883,0,1.48
0.01999999,0.12943,179684.3,0.6466664,1.48
0.03999998,0.12943,183232.8,1.273933,1.48
0.05999998,0.12943,188476.3,1.882381,1.48
0.07999997,0.12943,195363.7,2.472576,1.48
0.09999996,0.12943,203845.8,3.045066,1.48
0.12,0.12943,213874.7,3.60038,1.48
0.14,0.12943,225404,4.139035,1.48
0.1599999,0.12943,238388.8,4.661531,1.48
0.1799999,0.12943,252785.3,5.168352,1.48
0.1999999,0.12943,268551.2,5.659968,1.48
0.2199999,0.12943,285645.5,6.136836,1.48
0.2399999,0.12943,304028.2,6.599397,1.48
0.2599999,0.12943,323660.7,7.048082,1.48
0.2799999,0.12943,344505.6,7.483306,1.48
0.2999999,0.12943,366526.4,7.905473,1.48
0.3199999,0.12943,389687.8,8.314975,1.48
0.3399999,0.12943,413955.8,8.712192,1.48
0.3599999,0.12943,439297,9.097492,1.48
0.3799999,0.12943,465679.2,9.471233,1.48
0.3999999,0.12943,493071.3,9.833763,1.48
0.4199998,0.12943,521442.9,10.18542,1.48
0.4399998,0.12943,550764.7,10.52652,1.48
0.4599998,0.12943,581008.1,10.85739,1.48
0.4799998,0.12943,612145.5,11.17834,1.48
0.4999998,0.12943,644150.1,11.48965,1.48
0.5199998,0.12943,676995.9,11.79163,1.48
0.5399998,0.12943,710657.6,12.08455,1.48
0.5599998,0.12943,745110.7,12.36868,1.48
0.5799998,0.12943,780331.6,12.64428,1.48
0.5999998,0.12943,816297,12.91162,1.48
0.6199998,0.12943,852984.8,13.17094,1.48
0.6399997,0.12943,890373.4,13.42248,1.48
0.6599998,0.12943,928441.4,13.66647,1.48
0.6799998,0.12943,967168.8,13.90314,1.48
0.6999997,0.12943,1006536,14.13271,1.48

Figuur 2: Gedeeltelijke trainingsdata

Referenties

- [1] Y. Roh, G. Heo, and S. E. Whang, "A survey on data collection for machine learning," *IEEE Transactions on Knowledge and Data Engineering - Volume: 33, Issue: 4*, 2021.
- [2] A. Bhardwaj, D. Karger, H. Subramanyam, A. Deshpande, S. Madden, E. Wu, A. Elmore, A. Parameswaran, and R. Zhang, "Collaborative data analytics with datahub," *Proceedings VLDB Endowment*, 2015.
- [3] Google, "Google Fusion Tables: Data Management, Integration, and Collaboration in the Cloud," 2011.
- [4] Kaggle Inc., "Kaggle: Your Home for Data Science," 2025.
- [5] I. Terrizzano, P. Schwarz, M. Roth, and J. E. Colino, "Data wrangling: The challenging journey from the wild to the lake," *7th Biennial Conference on Innovative Data Systems Research*, 2015.
- [6] A. Halevy, F. Korn, N. F. Noy, C. Olston, N. Polyzotis, S. Roy, and S. E. Whang, "Goods: Organizing google's datasets," *Association for Computing Machinery*, 2016.
- [7] K. Maharana, S. Mondal, and B. Nemade, "A review: Data pre-processing and data augmentation techniques," *Global Transitions Proceedings*, vol. 3, no. 1, pp. 91–99, 2022. International Conference on Intelligent Engineering Approach(ICIEA-2022).
- [8] X. Ying, "An overview of overfitting and its solutions," *Journal of Physics: Conference Series, Volume 1168, Issue 2*, 2019.
- [9] Elite Data Science, "Overfitting in machine learning: What it is and how to prevent it," 2022.
- [10] J. Wei and K. Zou, "Eda: Easy data augmentation techniques for boosting performance on text classification tasks," 2019.
- [11] Y. Lu, L. Chen, Y. Zhang, M. Shen, H. Wang, X. Wang, C. van Rechem, T. Fu, and W. Wei, "Machine learning for synthetic data generation: A review," 2025.
- [12] T. Lee, S. McKeever, and J. Courtney, "Synthetic dataset generation using photo-realistic simulation with varied time and weather axes," *Electronics*, vol. 13, no. 8, 2024.
- [13] Blender, "Blender - The Free Open Source 3D Creation Software," 1994.
- [14] Unity, "Unity Real-Time Development Platform," 2005.
- [15] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 857–876, 2023.
- [16] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neuro-computing*, vol. 459, pp. 249–289, 2021.
- [17] T.-T. Wong and P.-Y. Yeh, "Reliable accuracy estimates from k-fold cross validation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1586–1594, 2020.
- [18] M. Kumar, "Top 10 ai simulation tools in 2025: Features, pros, cons & comparison," 2025.
- [19] Y. Li, "Deep reinforcement learning: An overview," 2018.
- [20] R. Nian, J. Liu, and B. Huang, "A review on reinforcement learning: Introduction and applications in industrial process control," 2020.
- [21] F. Torabi, G. Warnell, and P. Stone, "Behavioral cloning from observation," 2018.
- [22] Unknown, "Behavioral cloning in reinforcement learning," 2025.
- [23] A. Kumar, J. Hong, A. Singh, and S. Levine, "When should we prefer offline reinforcement learning over behavioral cloning?," 2022.

- [24] S. Kidera, K. Shintani, and S. Y. Toi Tsuneda, "Combined constraint on behavior cloning and discriminator in offline reinforcement learning," 2024.
- [25] V. G. Goecks, G. M. Gremillion, V. J. Lawhern, J. Valasek, and N. R. Waytowich, "Integrating behavior cloning and reinforcement learning for improved performance in dense and sparse reward environments," 2020.

8 Change-log

Versie	Wanneer	Wat
0.1	01-10-2025	Introductie geschreven
0.2	20-11-2025	Begonnen aan Trainingsdata
0.3	27-11-2025	Data discovery geschreven
0.4	29-11-2025	Data augmentation en generation geschreven
0.5	30-11-2025	Data generation begonnen
0.6	09-12-2025	Data generation en validatie geschreven
0.7	10-12-2025	Data generation en validatie afgeschreven
1.0	17-01-2026	Behavior cloning en conclusie toegevoegt